

Digital Twins for Manufacturing Systems: A Case Study Based On a Fischertechnik Factory

Lina Bilal^a, Alexander Hellwig^{b,*}, Pierre Treton^a, Didier Vojtisek^a, Jingxi Zhang^c, Benoît Combemale^a, Jean-Marc Jézéquel^a, Judith Michael^d, Bernhard Rumpe^b, Andreas Wortmann^c

^aIRISA, Université de Rennes, Rennes, 35042, Brittany, France

^bSoftware Engineering, RWTH Aachen University, Aachen, 52074, North Rhine-Westphalia, Germany

^cInstitute for Control Engineering of Machine Tools and Manufacturing Units (ISW), University of Stuttgart, Stuttgart, 70174, Baden-Wuerttemberg, Germany

^dProgramming and Software Engineering, University of Regensburg, Regensburg, 93040, Bavaria, Germany

Abstract

Digital twins for manufacturing systems connect physical production resources with synchronized models, data, and services to support monitoring, analysis, control, and optimization. This paper addresses the question of how a manufacturing digital twin reference architecture can be concretely instantiated and implemented across different operational use cases on a modular production system. While previous work has introduced digital twin definitions, conceptual models, and manufacturing-oriented reference frameworks, it has not sufficiently shown how such concepts can be mapped into a reusable, executable case-study environment that covers heterogeneous machine interfaces, synchronization, data and model management, and service-specific functionality. To address this gap, we use a Fischertechnik factory demonstrator as a compact manufacturing testbed and adapt a digital twin architecture consisting of gateways, a digital twin engine, synchronizers, data and model managers, and added-value services. The approach is investigated through three use cases, namely maximum energy-consumption monitoring, flexible replanning after production disruptions, and predictive maintenance, for which we derive stakeholder requirements, instantiate solution architectures, and implement the relevant communication, storage, synchronization, analysis, notification, and planning components. The resulting case study provides a concrete basis for comparing digital twin engineering approaches and shows how a shared manufacturing digital twin architecture can support multiple operational concerns beyond isolated monitoring scenarios.

Keywords: Digital Twin, Fischertechnik, Case Study

1. Introduction

Digital twins have become a central concept for connecting physical manufacturing systems with digital models, runtime data, and software services for monitoring, analysis, control, and optimization [1, 2]. In manufacturing, this integration is particularly relevant because production systems increasingly require flexible operation, transparent runtime observation, and the ability to react to changing conditions. However, the term digital twin is still used with different levels of integration, ranging from digital models and digital shadows to systems with bidirectional synchronization and feedback to the physical asset [3]. This makes it necessary to study not only conceptual definitions [4, 5] but also concrete architectures and implementations that show how digital twin systems can be engineered in practice.

Existing work provides important conceptual models and reference frameworks for digital twins. The five dimensional model emphasizes the interaction between physical entities, virtual models, data, services, and connections [1], while ISO 23247 defines a manufacturing-oriented framework for structuring digital twins around observable manufacturing elements and their digital representations [6]. In addition, the Asset Administration Shell provides a structured and semantically defined information framework for describing industrial assets and supporting interoperability in Industry 4.0 environments [7, 8]. Nevertheless, there remains a need for executable and reusable case studies that demonstrate how such concepts can be mapped to concrete manufacturing systems, heterogeneous communication interfaces, synchronization mechanisms, data and model management, and service-specific digital twin functionality.

This paper addresses this gap through a case study based on a modular Fischertechnik factory demonstrator. The demonstrator represents a compact manufacturing system consisting of storage, handling, transport, processing, and sorting modules, which are connected to a software stack through RevPi-based controllers and MQTT communication [9]. Based on this setup, we instantiate a digital twin reference architecture that distinguishes between the actual system, digital twin gateways,

*Corresponding author

Email addresses: lina.bilal@inria.fr (Lina Bilal), hellwig@se-rwth.de (Alexander Hellwig), pierre.treton@irisa.fr (Pierre Treton), didier.vojtisek@inria.fr (Didier Vojtisek), jingxi.zhang@isw.uni-stuttgart.de (Jingxi Zhang), benoit.combemale@irisa.fr (Benoît Combemale), jean-marc.jezequel@irisa.fr (Jean-Marc Jézéquel), judith.michael@ur.de (Judith Michael), rumpe@se-rwth.de (Bernhard Rumpe), andreas.wortmann@isw.uni-stuttgart.de (Andreas Wortmann)

a digital twin engine, synchronization mechanisms, data and model management components, and added-value services.

The core contributions of this paper are as follows. First, we provide a structured description of the Fischertechnik production system as a reusable manufacturing case study for digital twin engineering. Second, we show how a digital twin reference architecture can be instantiated for this demonstrator by mapping physical modules, communication interfaces, models, runtime data, and services to architectural elements. Third, we present three representative use cases: maximum energy consumption monitoring, flexible replanning after production disruptions, and predictive maintenance. These use cases cover different digital twin concerns, ranging from runtime observation and constraint monitoring to model-based disruption handling and maintenance-oriented analysis, which has been identified as a major application area for digital twins in manufacturing [10]. For each use case, we derive stakeholder-oriented requirements and describe how the digital twin architecture is adapted and implemented to realize the required functionality.

This paper is structured as follows: Section 2 introduces the Fischertechnik physical system, including its modules, control devices, interfaces, and twinable properties. Section 3 summarizes the digital twin reference models and architectural concepts used as the foundation of this work. Section 4 presents the three digital twin use cases and discusses their requirements, architecture, and implementation. Section 5 discusses the lessons learned from the case study and the reuse of architectural elements across the use cases. Section 6 concludes the paper and outlines future work.

2. Fischertechnik Physical System Description

This section explains the physical setup and control devices of the Fischertechnik production line, its control and communication interfaces, and the twinable properties of its modules.

2.1. Physical Setup and Control Devices

The demonstrator realizes a compact modular production island that combines storage, transport, processing, and quality inspection in a single reusable setup, as depicted in Figure 1. It includes the stations *HighBayWarehouse01*, *VacuumGripper01*, *VacuumGripper02*, *ConveyorBelt01*, *MultiProcessingStation01*, and *SortingLine01*. This composition supports the study of digital-twin synchronization, control integration, and adaptive services without requiring a full-scale factory.

The physical arrangement is complemented by the SysML structure model shown in Figure 2. The diagram separates a factory-level part view from reusable machine-system definitions. At the factory level, selected production modules are represented as typed parts, such as *warehouse: Warehouse* and *sortingLine: SortingLine*. Each part exposes typed ports for commands and state information, for example a warehouse command input and warehouse state output, or a sorting command input and sorting-line state output. This makes the interface between the factory structure and the digital representation explicit.

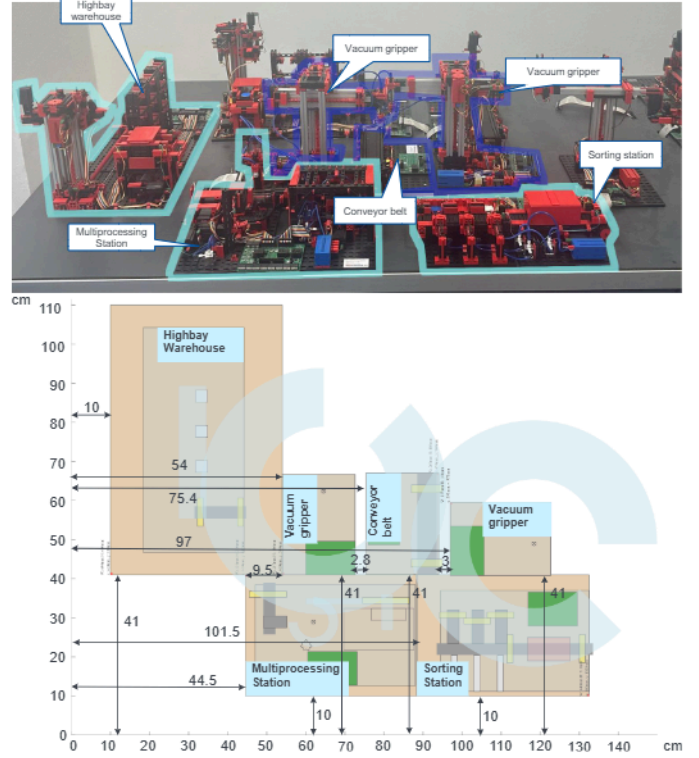


Figure 1: Fischertechnik demonstrator: (a) physical setup, (b) dimensioned top-view layout used for digital-twin synchronization.

The lower part of the model introduces reusable machine-system abstractions. The generic *System* definition groups hardware and software elements, while specialized systems refine this abstraction with concrete ports and properties. The *FactorySystem* specializes *System* with factory-level command and state interfaces.

For the digital twin, this model provides a structural reference for identifying represented modules, exposed command and state interfaces, and the integration of machine systems into the overall factory system. It therefore supports the mapping between physical modules, software interfaces, and digital twin services used in the following use cases.

Conceptually, the line represents a simplified manufacturing process in which raw material carriers are transformed into intermediate and final products through a sequence of handling and processing steps. The exact product composition and process path may vary depending on the concrete use case. In the demonstrator, this variability is represented in a simplified form so that the physical setup remains understandable while still reflecting the core characteristics of a modular production environment.

In the reference scenario used throughout this paper, a carrier is retrieved from *HighBayWarehouse01*, transferred by a vacuum gripper to *ConveyorBelt01*, processed in *MultiProcessingStation01*, returned to the conveyor, and finally inspected and routed by *SortingLine01*.

VacuumGripper01 primarily mediates between storage and transport, whereas *VacuumGripper02* mediates between transport and downstream processing/sorting steps. The *Conveyor-*

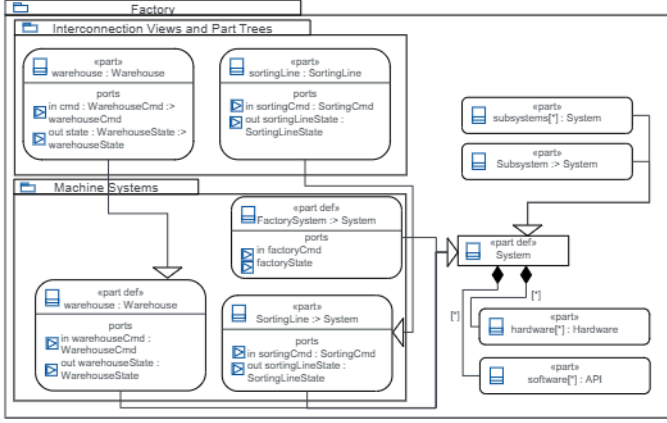


Figure 2: SysML structure model of the Fischertechnik factory system. The diagram combines a factory-level part and interconnection view with reusable machine-system definitions, including command and state ports for selected modules and a generic system abstraction for hardware and software elements.

Belt01 provides the main transport path between stations and serves as the shared transport backbone of the line. In addition, the conveyor acts as an intermediate handover point between handling and processing operations.

The *MultiProcessingStation01* represents the main processing unit in the line. In the demonstrator scenario, it is used to model preparation and transformation steps such as mixing, heating, and forming. After processing, the intermediate product is returned to *ConveyorBelt01* and transported to the next handling point. This enables a cyclic interaction between transport and processing, which is typical for modular production systems.

At the end of the line, the product is transferred to *Sort-ingLine01*, which represents the quality inspection and sorting stage. Here, product properties are checked and the product is routed according to the inspection result. In the simplified demonstrator setting, this station captures the essential idea of quality-dependent branching in a production process, including the distinction between accepted and rejected parts.

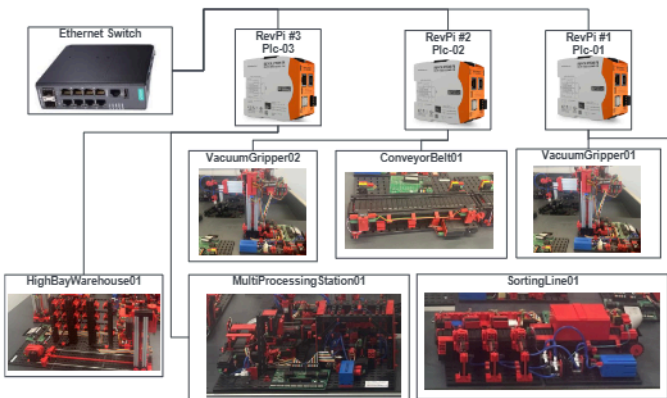


Figure 3: Cabling and controller assignment of the Fischertechnik demonstrator, showing the RevPi-based PLC nodes (Plc-01 to Plc-03), Ethernet switch, and connections to the physical stations.

The physical setup is controlled by three RevPi-based PLC

nodes, labeled *Plc-01*, *Plc-02*, and *Plc-03*, that coordinate the stations as a production island (cf. Figure 3). The RevPis are connected through an Ethernet switch and wired to the Fischertechnik stations, where they acquire sensor and state information and execute actuator commands.

2.2. Control and Communication Interfaces

At the control layer, stations expose low-level digital I/O signals. At the communication layer, these signals are made available via sockets or MQTT. At the digital-twin layer, higher-level commands and derived states are represented as machine-readable properties and methods. High-level actions such as *pickup(x,y,z)* are decomposed by the control layer into sequences of low-level actuator commands (e.g., vertical, horizontal, and rotational motions), which are then exposed to the digital twin through MQTT topics.

The highbay warehouse consists of nine slots, an entry and output point, and a transport arm. This results in a payload being able to be in one of the nine slots, on the arm, or in the intake. The sorting station is able to recognize three different colors, green, red, and yellow, in its intake. The outlets have push mechanisms to move payloads down the ramps. Conveyors transport the payload between the intake and the push arms. The multiprocessing station consists of an intake, a turntable, and an oven module. The turntable can rotate 270 degrees and has three stations: one for processing the payload, one to receive the payload from the intake, and one for transporting it to the oven module with a vacuum gripper. The oven consists of a door, an LED lamp, and a tray that moves payloads in and out. The conveyor belts consist of a belt and two sensors, with light barriers at the ends of the belt to recognize any payload.

Each machine may show deviations that are machine-specific. As an example, the vacuum gripper shows a slight deviation due to a machine-specific motion profile. These profiles are commonly modeled as trapezoids or S-curves [11]. The Unity simulation in Figure 1 reflects the physical machine control by representing each movable axis as a Boolean command. Payload recognition is simulated with light barriers that emit a value upon collision with a payload. Each machine itself is composed of axis motors for each axis. The vacuum module is a unit itself. The conveyor belt can be taken as a belt connected to motors for movement. While the physical system shows clear structures, the virtual model contains multiple objects, which are connected through a reference as a whole entity to be moved. On a shop floor, there are multiple roles [12]. The standard defines roles such as maintenance technician, supervisor, engineer, and manager alongside their competencies. For our Fischertechnik example, we keep a simpler setup. The engineer and manager roles are provided by supervisors defining use cases of the production line. The supervisor must be a person familiar with the production processes. Finally, the maintenance technician is responsible for emergency exits and for fixing physical machines when they encounter obstacles, such as a machine getting stuck.

2.3. Twinable Properties of Physical Modules

The properties that can be twinned cover the complete set of capabilities exposed by each physical station, including both (i) raw signals as used by the control software and (ii) derived, preprocessed properties for ease of use.

The vacuum gripper provides three degrees of freedom: a vertical axis, a horizontal axis, and a rotational axis. Each axis is commanded via dedicated actuator signals (e.g., `move-vertical-up/down`, `move-horizontal-forward/backward`, and `move-rotation-clockwise/counterclockwise`), while reference switches and encoder impulses report the axis state. Topics such as `motor-vertical-pos` expose preprocessed positions. The gripper also includes pneumatic equipment controlled via compressor and valve commands (e.g., `enable-compressor` and `enable-valve`) and a normalized suction property in `[0, 1]`.

The conveyor belt exposes two directions of motion and a small set of sensors. Concretely, it is commanded through `move-conveyor-forward` and `move-conveyor-backward`, while phototransistors (e.g., at the feed and swap stations) publish Boolean detection signals. The sorting line extends this setup by adding sensing and actuation for quality decisions: a color sensor publishes an integer reading (topic `color-sensor`), and multiple light barriers delimit the measurement and routing sections (e.g., `light-barrier-inlet` and `light-barrier-behind-color-sensor`). Sorting is executed via three pneumatic ejectors controlled by separate valve signals (e.g., `enable-valve-first-ejector` to `enable-valve-third-ejector`).

The most complex station is the multi-processing station, which combines a turntable, a heating module, and a vacuum-based transport mechanism. Its turntable rotates to discrete positions (e.g., `vacuum`, `belt`, `saw`), is controlled via `move-turn-table-clockwise` and `move-turn-table-counterclockwise`, and is monitored through reference switches. The heating module provides a mocked temperature value and exposes an LED indicator (`enable-oven-light`) as a visible proxy for the heating process. Material flow is finalized through an output conveyor, while light barriers at the oven and conveyor end provide feedback for process coordination.

Finally, the high-bay warehouse comprises nine storage bays, an elevator mechanism, and an instrumented exit. Its motion is controlled via horizontal and vertical actuators (e.g., `move-arm-to-rack`, `move-arm-to-conveyor`, `move-vertical-up/down`) and a cantilever (e.g., `move-cantilever-forward/backward`). Encoder impulses and virtual position topics (e.g., `horizontal-enc-`, `vertical-enc-`, `horizontal-pos`, `vertical-pos`) allow the twin to track the current pose. Based on these signals and the exit sensors (e.g., `light-barrier-inside/outside`), the warehouse maintains an internal occupancy model of which bays are currently filled.

Inter-station coordination requires a digital twin system that integrates these heterogeneous signals into a consistent digital

shadow. Each RevPi publishes raw I/O states via MQTT; downstream components can then compute derived properties (e.g., discrete positions from encoder impulses) and expose them as higher-level, machine-readable twin properties.

3. Reference Architecture and Glossary

Reference models for digital twins provide different perspectives on their structure. Some clarify conceptual elements, while others focus on implementation-oriented architectures and interoperability. We briefly introduce two such perspectives to position the terminology used in this paper; the concrete realization of the use cases is then based on the reference architecture introduced at the end of this section.

A widely used conceptualization is the 5D model from Tao et al. [1]. It describes a digital twin as the integration of a *physical entity*, a *virtual model*, *digital twin data*, *services*, and *connections*. The model is useful for emphasizing that a digital twin is not only a model of a physical system, but an integrated system of physical, virtual, data-driven, service-oriented, and connective elements.

Another popular reference model coming from industry is the ISO 23247 standard which can be considered an implementation-oriented architecture that improves interoperability. ISO 23247 defines a digital twin framework for manufacturing systems [6]. In this standard, the twinned elements are described as *observable manufacturing elements* (OMEs), which may refer to manufacturing resources such as machines, robots, production lines, or human operators. These elements expose observable properties that can be represented and processed within the digital twin framework.

The functional view of ISO 23247 structures the digital twin framework into several entity layers. The *device communication entity* collects and processes data from OMEs and, where applicable, enables control or actuation. The *digital twin entity* uses this communication layer to represent, manage, operate, simulate, and maintain the observed manufacturing elements. Finally, the *user entity* provides access for users or external applications that analyze, monitor, or control the manufacturing system. ISO 23247 therefore provides an implementation-oriented view on digital twins in manufacturing, with a particular focus on the interaction between observable manufacturing elements, communication mechanisms, digital representations, and user-facing functionality.

Reference Architecture for Digital Twin Systems. The discussed reference models emphasize different aspects of digital twins, such as conceptual dimensions and manufacturing-specific functions. For this paper, however, we require an architecture that describes the internal structure of a digital twin system and distinguishes its core components from external systems and added-value functionality. We therefore use the reference architecture shown in Figure 4 as the conceptual foundation for realizing our use cases [5]. This architecture provides the terminology and structural frame used in the remainder of the paper to position our contribution.

Digital twins are software-intensive systems that represent, observe, and potentially control an actual system through a set

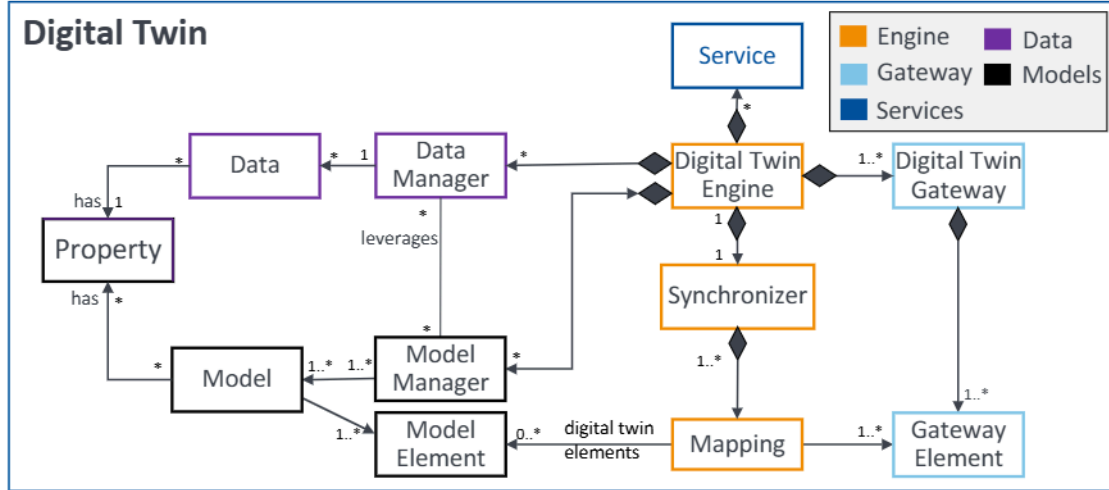


Figure 4: The main DT concepts are a gateway connecting to the Physical Twin, models and data representing it, an engine synchronizing properties of interest between both, and added-value services.

of digital artifacts and runtime connections. The selected reference architecture is particularly suitable for our work because it makes the boundary of the digital twin explicit. The actual system itself is not part of the digital twin. Instead, the digital twin interacts with the actual system through dedicated interfaces that enable data acquisition, observation of changes, and, where applicable, the execution of control commands.

At the center of the architecture is the *Digital Twin Engine*. The engine coordinates the interaction between gateways, data, models, and services. Its main responsibility is to synchronize relevant properties between the actual system and its digital representation. For this purpose, the architecture introduces *mappings* between gateway elements and model elements. Such mappings specify which elements correspond to each other and in which direction synchronization takes place, for instance from the actual system to the digital twin, from the digital twin to the actual system, or bidirectionally. Synchronization may be triggered periodically or by specific events. In this paper, we use this notion of mappings and synchronization to reason about how changes in one part of a digital twin system can affect other parts.

The *Digital Twin Gateway* connects the digital twin to the actual system. It exposes selected features of the actual system as gateway elements, such as readable properties, observable events, or invokable functions. This abstracts from the concrete communication technology and separates technology-specific concerns from the internal representation and coordination logic of the digital twin.

The architecture further distinguishes between *data* and *models*. Data denotes runtime or historical information collected, processed, and managed by the digital twin. A data manager provides controlled access to such data and its associated metadata, such as origin, update time, precision, or processing state. Models, in contrast, provide structured digital representations of relevant aspects of the actual system. They conform to modeling languages and are managed by model managers, which are responsible for maintaining the

consistency and integrity of model elements and their relations. This separation is central for our paper, because it allows us to discuss digital twin systems not only as data-driven systems, but also as model-based systems whose models need to be managed, related, and kept consistent.

Finally, the architecture includes *services* as added-value functionality built on top of the digital twin core. Services may use the available data and models for monitoring, analysis, simulation, prediction, optimization, or decision support. However, services interact with the actual system only through the digital twin engine and the gateway infrastructure. This preserves the architectural boundary of the digital twin and allows access to models, data, and control functions to be restricted and coordinated.

Overall, we use this reference architecture as the baseline for describing digital twin systems as compositions of gateways, models, data, synchronization mechanisms, and services. In the remainder of this paper, the architecture serves as an analytical frame to clarify which architectural elements are addressed by our approach, which elements are assumed as surrounding infrastructure, and how our contribution relates to the core functionality of a digital twin system.

4. Digital Twin Use Cases

In this section, we present three use cases for digital twins built on the Fischertechnik production line. Each use case presents a problem from the production domain, introduces a set of user stories and requirements for the digital twin, presents a solution architecture, and discusses an implementation of the digital twin.

With these use cases, we aim to demonstrate the specific problems digital twin engineers must solve when creating a digital twin for the manufacturing domain. Furthermore, we illustrate how practitioners can adapt the reference architecture from [5] to fit specific scenarios. We also discuss how effectively

this architecture resolves common problems within the manufacturing domain. Additionally, future researchers can use the provided user stories and requirements to compare alternative development methodologies and architectures for digital twins.

The use cases are ordered from least complex to most complex, such that parts of the architecture from the earlier use cases can be reused in the later ones. Additionally, the use cases are structured such that the digital twins of the production line can run in parallel. Thus, they serve as the basis for a discussion of the federation and interaction of multiple digital twins.

4.1. Use Case 1: Monitoring Maximum Energy Consumption

Motivation. Regulations or available capacity can limit the peak consumption of a resource, e.g., the maximum power a factory can use at any one time. In a factory, exceeding this capacity can stop production, and failing to comply with regulations may lead to fines or a shutdown. Thus, the digital twin should enable measurement, logging, and management of scarce resource usage.

In this example, we focus on a production line connected to a smart energy grid, which imposes a dynamic limit on the total power usage. To ensure this constraint holds, the physical twin must measure the consumption data of the individual machines and synchronize it with the digital twin. The digital twin then aggregates the consumption data and compares it with the target value. To enable long-term auditing, the digital twin permanently stores both the individual and aggregated measurement series in digital shadows.

Stakeholders. The main stakeholders are the *factory manager* and the *operator*.

User stories. As a *operator*, I want to

- **US1** monitor the current energy consumption of each machine because I want to detect anomalies.
- **US2** monitor the current aggregated energy consumption of the production process because I want to plan the next production steps.
- **US3** receive a notification if the current energy consumption of the production process exceeds the limit set by the smart energy grid because I want to avoid a shutdown.

As a *factory manager*, I want to

- **US4** avoid exceeding the energy consumption limits that the smart grid imposes because I want to avoid a production shutdown.

Fulfillment Criteria/Requirements. Next, we convert the user stories into smaller, independent requirements, which form the basis for a solution implementation.

RQ1 Machine Logging. The system must measure and log the energy consumption of each machine.

RQ2 Process Aggregation. The system must calculate and log the current energy consumption of the overall production process.

RQ3 Limit Retrieval. The system must obtain the energy consumption limit from the smart grid.

RQ4 Budget Calculation. The system must calculate and log the difference between the energy consumption limit and the total production consumption.

RQ5 Dashboard. The system must provide a live visualization of the energy consumption of each machine, the aggregated production consumption, and the difference between the limit and the aggregated consumption.

RQ6 Notifications. The system must notify the operator and the factory manager if the aggregated energy consumption approaches or exceeds the limit.

US1 (Machine Monitoring). necessitates the measurement of energy usage on the machine level, leading to **RQ1 (Machine Logging)**. Since the operator needs to manually check the current and historical state of this consumption, the system needs a dashboard (**RQ5 (Dashboard)**). Furthermore, **US2 (Process Monitoring)** requires that the operator can observe the current aggregated energy consumption of the production process. This directly implies the continuous aggregation of machine-level measurements (**RQ1 (Machine Logging)**) into a process-level metric, resulting in **RQ2 (Process Aggregation)**. In addition, planning future production steps necessitates access not only to instantaneous values but also to historical trends, which the logging aspect of **RQ2 (Process Aggregation)** and the visualization requirement in **RQ5 (Dashboard)** cover.

US3 (Limit Alerting) introduces the constraint that the system must react to external energy limits from the smart grid. Consequently, the system must retrieve and update the current limit dynamically, which **RQ3 (Limit Retrieval)** formalizes. To determine whether the aggregated consumption approaches or exceeds the limit, the system must continuously compare the aggregated consumption with the limit, leading to **RQ4 (Budget Calculation)**. Based on this comparison, the system must send timely notifications to the operator, which **RQ6 (Notifications)** specifies. The live aspect of the user story further reinforces the need for visualization, as **RQ5 (Dashboard)** defines.

US4 (Limit Compliance) from the factory manager's perspective focuses on preventing production shutdowns that occur when production exceeds smart grid limits. This objective aligns with proactive monitoring and early intervention. Therefore, the notification mechanism in **RQ6 (Notifications)** must not only trigger upon limit violations but also when the consumption approaches a critical threshold. Moreover, the continuous acquisition of the grid limit (**RQ3 (Limit Retrieval)**) and the comparison logic (**RQ4 (Budget Calculation)**) are essential to support strategic decision-making at the management level.

4.1.1. Digital Twin Design to Fulfill the User Story

We solve the user stories through a digital twin of the fischertechnik production line. In a first step, we create a functional model of the machines and the production line using a MontiArc Component and Connector diagram [13]; Figure 5 shows the result. Since the main focus is on energy consumption, we leave most aspects related to production vague via underspecification. The top-level function *ProductionLine* receives material as input and outputs the finished product. Additionally, it has an output port for the aggregated energy usage of the production line, contributing to fulfilling *RQ2 (Process Aggregation)*. The material from the input passes through multiple transformations, such as the *Drilling* function, as well as functions to model the transport between the transformation steps. The transport components are necessary because the physical components implementing the functions, such as a vacuum gripper, will also use energy. Each of the internal components measures and outputs its current energy usage as data via a sensor. All of these data points feed into the *sum* component, which aggregates and forwards them to the output of *ProductionLine*. Thus, this also provides the basis for fulfilling *RQ2 (Process Aggregation)*.

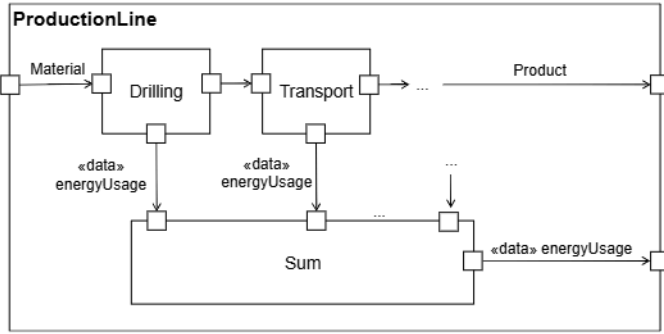


Figure 5: High-level C&C model of the production line with a focus on energy consumption

From this architecture of the CPS, we can derive the first digital shadows to implement the user stories. The system can observe the channels streaming data between the transformation steps and transport functions to the *Sum* function and receive the digital shadows of the energy consumption of each machine (*RQ1 (Machine Logging)*). Additionally, the system needs to save the maximum energy that the smart energy grid provides in a digital shadow, in relation to *RQ3 (Limit Retrieval)*.

The channel between the *Sum* and the top-level function represents the shadow of the total energy consumption (*RQ2 (Process Aggregation)*). Since it is an aggregation of the energy consumption shadows of each machine, a service needs to calculate it, which we will call *shadow aggregator* from now on. The last shadow of this digital twin is the difference between the current usage and the limit, as required by *RQ4 (Budget Calculation)*, and the same *aggregator* service also calculates it.

Based on these shadows, we can design the other services of the digital twin. To fulfill *RQ5 (Dashboard)*, the dashboard needs to provide access to all digital shadows such that the

operator and factory manager can manually inspect and plan based on energy consumption and availability. As an easy-to-understand visual representation of the shadows, the dashboard presents multiple line charts. To allow the focus on a subset of the shadows, users can dynamically toggle the visibility of the charts.

To fulfill requirement *RQ6 (Notifications)*, we add a notification service, which observes the difference between the available and used energy and sends notifications, such as emails or push notifications, to the operator and the factory manager. The service sends these messages when consumption exceeds configured limits.

4.1.2. Example Architecture of the DT Demonstrator

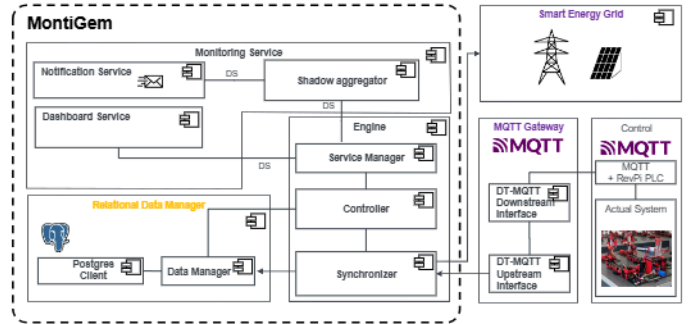


Figure 6: Architecture of the digital twin for the energy monitoring use case

For the implementation in the fischertechnik demonstrator, we use the architecture displayed in Figure 6. RevPI PLCs control the actual twin, the fischertechnik production line, and their control software contains an MQTT gateway, which can send control commands to or receive data from the production line. The PLCs proactively send all sensor data and the current state of the machines to the MQTT broker, which the figure omits. The gateway of the digital twin also uses MQTT, and its *DT-MQTT Downstream Interface* subscribes to all relevant energy sensor traces from the actual system. The *DT-MQTT Upstream Interface* translates the data types that MQTT uses into the data types of the digital twin.

Afterwards, the gateway passes all data from the physical systems to the *Synchronizer* of the DT-Engine, which synchronizes the incoming shadow data with the data management and the services. Additionally, it polls the REST interface of the smart grid to retrieve the currently available energy. The *Data Manager* serializes data into a Postgres database and provides a cache for recently used data.

The *Service Manager* initializes and connects the services of the system. The *Shadow Aggregator* observes the incoming digital shadows via the *Service Manager* and creates the two processed shadows, the aggregated energy usage and the difference to the available energy.

The *Dashboard Service* accesses the raw and processed digital shadows via the *Data Manager* and provides a real-time visualization of machine-level consumption, aggregated usage, and the remaining energy budget. It offers interactive line charts and configurable filters so that operators and factory managers

can focus on specific machines or time intervals for detailed analysis.

The *Notification Service* subscribes to the aggregated consumption and difference shadows and continuously evaluates them against configurable warning and limit thresholds. Whenever the remaining energy budget falls below a defined margin or the aggregated consumption exceeds the limit, the service automatically dispatches alerts, such as emails or push notifications, to the relevant stakeholders.

Together, these three services comprise the overall monitoring service of the digital twin. Finally, the *Controller* orchestrates the *Synchronizer* and *Service Manager*.

4.1.3. Example Implementation in the DT Demonstrator

To implement this architecture, we use the MDE generator framework MontiGem [14], which generates a full web application comprising a backend, database, and frontend. We use class diagrams to define the data structures used across all three of these components. In addition, we use GUI-DSL models to define the pages of the frontend. We write handwritten interfaces to and from other systems in Java with the help of Spring Boot. We can also use Java to add behavior to GUI pages and implement additional services.

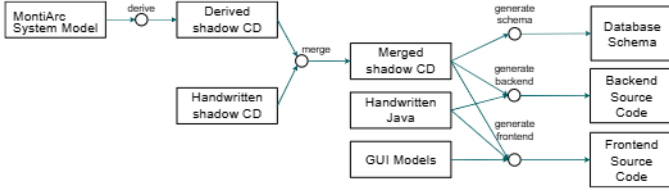


Figure 7: Generation of the Digital Twin using MontiGem as a formalism transformation graph

Figure 7 shows the artifacts that serve as inputs in the generation process of the digital twin. To be usable in the rest of the system, we have to define the digital shadows in the class diagram. In this simple use case, the system saves all values as a time series of doubles. While we model the shadow from the smart power grid manually, the generator can automatically derive all other shadows from the functional model of the CPS. A DT-specific generator reads the MontiArc model, extracts the function structure, and converts all connectors between components into a class diagram of the digital shadows. The generator automatically merges this diagram with the handwritten shadow definition and uses it to generate the base data structure of the MontiGem application.

To fill the digital shadows with data, we need to implement the gateways to the external systems. By augmenting the MontiArc model via model tagging [15], we can define the connectors that serve as data sources for each shadow. We enrich each outgoing port related to energy measurement of the transformation and transport steps with the MQTT topic used to broadcast the sensor values. A generator then derives the transformation from MQTT data types into the DT data types. Thus, the generator implements the *DT-MQTT Upstream Interface*. We need to manually implement and configure the *DT-MQTT Downstream Interface*, which connects the DT to the MQTT broker.

The generated MontiGem application contains a mechanism to automatically synchronize data changes into the database and connected frontend clients. This infrastructure takes on the role of the *Synchronizer*. We add a handwritten extension to the *Synchronizer*, which polls the REST API of the smart energy grid and writes the returned data into the appropriate digital shadow.

MontiGem automatically generates the *Data manager* from the merged class diagram describing the digital shadows, including a schema for the Postgres database, all DAOs needed to translate between the database types and the DT types, and a caching layer for frequently used data. MontiGem provides the *Postgres Client* as a configurable runtime component.

Based on these generated artifacts, we implement the three services from the architecture. First, we implement the *Shadow aggregator* by adding an observer to all digital shadows for the individual power consumption of the machines. On each update, the observer calculates the aggregated power consumption and writes it into the corresponding digital shadow. Additionally, the shadow aggregator reacts to changes in the digital shadow of the energy currently available from the smart grid. It calculates the difference between the used and available energy and saves the result as a digital shadow.

Second, we extend the automatically generated base dashboard to display the energy difference digital shadow as a line chart. The generator derives all other digital shadows from the MontiArc model, and thus the dashboard already contains the corresponding charts for the factory manager and operator to manually monitor these values.

Lastly, we implement the notification service. It watches the energy difference shadow for new values and sends an email to the operator and factory manager once the energy difference crosses the warning threshold. It sends an error email when the system uses more energy than the smart grid can provide, and thus the difference shadow falls below zero. We extend the dashboard with a configuration page to set email addresses and the warning limit.

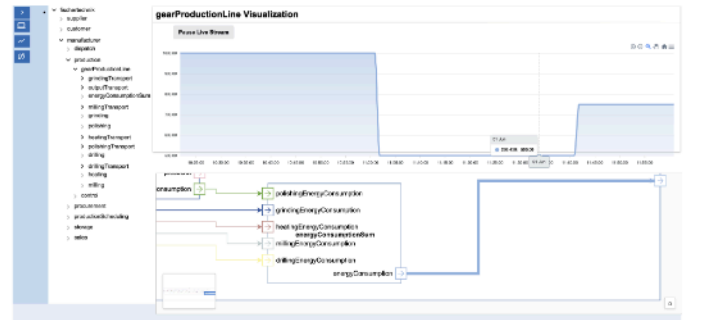


Figure 8: Dashboard Service: Visualization of the energy usage digital shadows

4.1.4. Conclusion

This use case shows how a digital twin can monitor a resource constraint by combining machine-level measurements, externally provided limits, derived digital shadows, and services for visualization and notification. The functional

model provides the basis for deriving most digital shadows, while handwritten extensions connect the demonstrator to the smart grid and implement the service logic. Together, these elements fulfill the monitoring, aggregation, limit retrieval, budget calculation, dashboard, and notification requirements for the energy monitoring scenario.

4.2. UC2: Flexible Replanning

Motivation. Unexpected failures or temporary unavailability of critical components in a factory can disrupt production and cause unplanned downtime. Since production steps are interdependent, a local disruption in a component may prevent subsequent steps from being executed as planned, even if the rest of the production line is operational. To reduce such disruptions, whenever possible, the DT must be able to flexibly replan the factory's operation by choosing an alternative way to achieve a downgraded production goal when the nominal path is disrupted.

In the fischertechnik factory, the production process relies on successive transfer and forwarding actions to move a workpiece between the factory's stations. We consider a scenario where a component failure prevents the nominal path from progressing as expected. The DT monitors the execution to detect such faults, records when and where they occur, and switches to an alternative execution path that advances the workpiece, thereby avoiding unnecessary downtime.

Stakeholders. The main stakeholders are *the factory manager* (minimize downtime, meet production targets) and *the maintenance engineer* (repair faulty components).

User Stories. As a *maintenance engineer*, I want to

- **US1** be notified when the DT detects that the nominal production path is disrupted and when human intervention is required to restore the system, so that I can intervene for maintenance operations.

As a *factory manager*, I want to

- **US2** analyze when and where replanning was triggered, so that I can understand recurring disruptions and evaluate the chosen strategies.
- **US3** minimize downtime so that the production targets can be met.
- **US4** analyze records of component downtime and maintenance duration so that I can make informed reinvestment decisions.

Fulfillment Criteria/Requirements.

RQ1 State Tracking. The system must continuously monitor the progress of the nominal production path during operation and keep track of the current execution state.

RQ2 Disruption Detection. The system must detect disruptions when the expected progress condition for the current step is not met.

RQ3 Context Recording. The system must record the disruption context, including the timestamp, the affected step, and the active nominal path.

RQ4 Path Selection. The system must select an alternative execution path when a discrepancy is detected, so that the production cycle can continue toward the intended objective.

RQ5 Strategy Logging. The system must record the alternative strategies and when they were applied, so that recovery actions can be analyzed.

RQ6 Maintenance Notification. The system must notify the maintenance engineer when a disruption in the nominal path is detected, including information about the affected component.

RQ7 Downtime Recording. The system must record the duration of each disruption, from disruption to recovery, in order to quantify downtime.

RQ8 KPI Calculation. The system must compute indicators, such as mean time between failures (MTBF) and mean time to repair (MTTR), for each component in order to analyze downtime impact and justify component replacement.

US1 (Maintenance Alert). requires data to detect deviations and to support intervention. This implies maintaining a continuously updated representation of the current execution state and the expected nominal progression at that state (**RQ1 (State Tracking)** and **RQ2 (Disruption Detection)**), and capturing the deviation context that can be communicated in the notification, including when and where it occurred and the affected component (**RQ3 (Context Recording)** and **RQ6 (Maintenance Notification)**). **US2 (Replanning Analysis)** requires traceability data to support the analysis of replanning, which means the system must persist historical records that locate each deviation in time and along the nominal path and must also persist which alternative fallback strategy was selected and when it was applied, as captured by **RQ3 (Context Recording)** and **RQ5 (Strategy Logging)**. **US3 (Downtime Reduction)** focuses on keeping the production cycle running to reach a downgraded objective when the nominal path is disrupted. This requires that the system first maintain an up-to-date execution state (**RQ1 (State Tracking)**) and detect the disruption (**RQ2 (Disruption Detection)**), so that a replanning mechanism can react to the detected anomaly by choosing an alternative strategy rather than stopping the production cycle (**RQ4 (Path Selection)**). **US4 (Maintenance Analytics)** requires downtime measurements for each component that are comparable over time. This necessitates recording, for each deviation, the affected component, the deviation's start timestamp, and the recovery timestamp, so that each disruption duration can be computed, and then aggregating these durations per component in line with **RQ7 (Downtime Recording)**, while also providing the required inputs to compute component reliability indicators in **RQ8 (KPI Calculation)**.

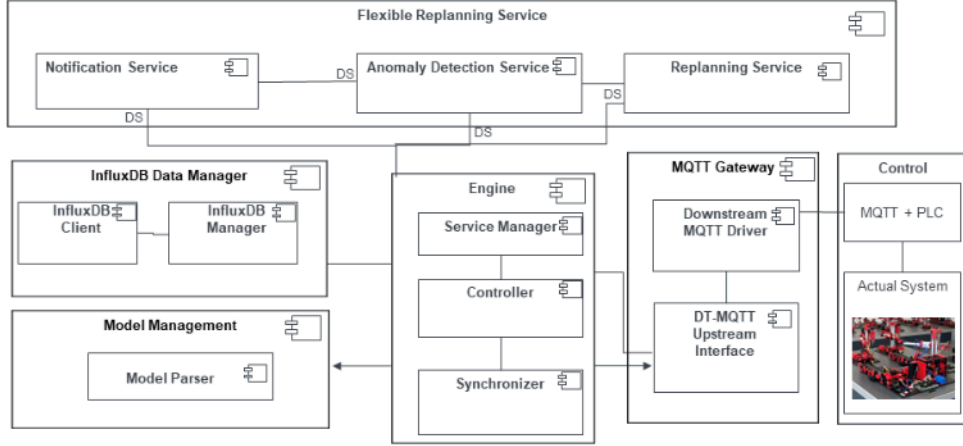


Figure 9: Digital Twin Architecture for the flexible replanning use case

4.2.1. Digital Twin Design to Fulfill the User Story

We model the production cycle as the progression of a workpiece across the FT demonstrator stations using a SysML v2 state machine (Figure 10). In our scenario, a conveyor belt (CB) failure prevents the workpiece from progressing along the nominal path as expected. When the disruption is detected, the DT triggers a predefined recovery strategy that selects an alternative mission based on the current workpiece location and nearby available resources. If neither the nominal path nor a recovery strategy can be executed, for example because nearby resources are blocked, execution stops until the faulty component is repaired and recovery is confirmed.

The DT uses this behavioral model as a runtime reference for interpreting the current production state. During operation, it continuously observes runtime feedback and checks it against the expected workpiece progression derived from the state machine. This fulfills *RQ1 (State Tracking)* and *RQ2 (Disruption Detection)* because feedback is interpreted as runtime evidence of whether the current production step can progress according to the nominal model.

Based on this comparison, the DT identifies the disrupted production step and affected part of the nominal path, records the disruption context for later analysis (*RQ3*), alerts the maintenance engineer (*RQ6*), and lets the replanning component select an alternative SCADA mission so the cycle can continue despite the disruption (*RQ4*).

The selected strategy, its activation time, and the later recovery confirmation are recorded. When the faulty component is repaired, the DT associates the recovery timestamp with the initial disruption timestamp, allowing the disruption duration to be computed. These records support the analysis of replanning decisions and the computation of downtime KPIs, as required by *RQ5 (Strategy Logging)*, *RQ7 (Downtime Recording)*, and *RQ8 (KPI Calculation)*.

4.2.2. Example Architecture of the DT Demonstrator

The architecture shown in Figure 9 is instantiated on the Fischertechnik demonstrator to support runtime monitoring of the nominal production path, disruption detection, and flexible

replanning when a component becomes unavailable.

To instantiate the architecture, we configured the DT using several SysML v2 models describing the production resources and the nominal progression of the workpiece. The machine-level models capture the structural and behavioral descriptions of each resource. In addition, a dedicated behavioral model (Figure 10) describes how the workpiece is expected to progress along the nominal production path.

The *Model Management* component handles these SysML v2 models. The *Model Parser* loads the models and provides the relevant model information needed by the DT components at runtime. This allows the DT to rely on a model-based description of the production resources and their expected behavior during execution.

The *MQTT Gateway* provides the communication interface through which runtime feedback is made available to the DT. The *Downstream MQTT Driver* captures the MQTT topics published during operation, while the *DT-MQTT Upstream Interface* exposes the corresponding data on the DT side according to the variables expected by the model. This allows feedback received from the PT to be interpreted as runtime data that is consistent with the SysML v2 model.

The *DT Engine* coordinates runtime execution. Its *Service Manager* provides services with interpreted model information and runtime data, the *Controller* routes observations, and the *Synchronizer* maintains the correspondence between received runtime data and the interpreted behavioral state.

The *InfluxDB Data Manager* records timestamped feedback and relevant events through the *InfluxDB Manager*. These records are used to analyze when disruptions occurred, which component was affected, which recovery action was applied, and when the system returned to the nominal path. They also provide the historical data required to calculate downtime-related indicators.

The flexible replanning component of the architecture is organized around the *Anomaly Detection Service*, the *Notification Service*, and the *Replanning Service*. The *Anomaly Detection Service* uses the behavioral models as references to identify disruptions in the observed execution. The *Notification Ser-*

vice makes disruption and recovery information available to the maintenance engineer. The *Replanning Service* uses the detected disruption context to select an alternative mission when the nominal production path is disrupted.

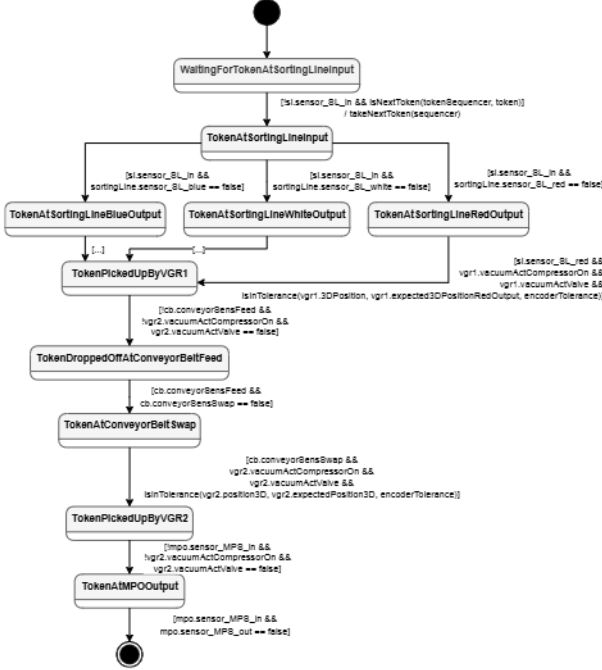


Figure 10: State machine of the workpiece's location on the FT demonstrator

4.2.3. Example Implementation of the DT Demonstrator

The implementation of this architecture is based on a Java/Spring prototype. At startup, the DT Engine loads the SysML v2 models, including the machine-level models of the FT demonstrator and the model for the workpiece location across the production cycle. It also initializes the gateway configuration, checks the database connection, and activates the runtime services with the parsed model information.

The SysMLV2Parser implements the model management component of the architecture. It parses the SysML v2 files and produces parsed model objects that contain both structural and behavioral information. From the structural descriptions, the SysMLStructuralParser extracts elements such as parts, attributes, constraints, and actions. From the behavioral elements, the SysMLBehavioralParser extracts states, transitions, and guards. In particular, the behavioral model is used to initialize the current interpreted state, while the structural model provides the attributes and constraints used to interpret incoming feedback values. These parsed elements are then used by the runtime services as a reference for interpreting runtime feedback.

During execution, runtime feedback enters the DT via MQTT messages published by the PLC. *MqttInboundConfig* forwards messages to *MqttMessageHandler*, while *GatewayServiceConfiguration* defines how PLC topics map

to DT topics. The handler applies the matching rule, republishes the payload through *MqttOutboundConfig*, extracts a *DataRequest* with target topic, model name, attribute, value, and timestamp, and forwards it to the DT Engine for service dispatch.

The data management component is implemented by the *DigitalShadow*, which uses *InfluxDBClient* configured in the *InfluxDBConfig* to persist the runtime feedback messages. For each message selected for storage, the *DigitalShadow* records the source topic, payload, extracted value, and the timestamp in the corresponding *InfluxDB* measurement. This storage is used to keep a history of the observed execution and to later analyze disruptions, selected recovery actions, and recovery duration. In this way, the implementation provides the runtime data needed to later compute indicators such as MTTR, MTBF, and disruption duration.

The anomaly detection service maintains the current interpreted state of the production cycle. For each dispatched feedback object, it updates the corresponding model attribute and evaluates outgoing transitions using the conditions extracted from the SysML v2 state machine. If a transition is enabled, the interpreted state progresses; if the expected progression cannot be validated or a timing constraint is violated, the service reports an anomaly. In the scenario considered, the service monitors whether the workpiece departs from the feed station within the expected time window. If the workpiece does not depart or if it fails to reach the swap station within the expected time, the service concludes that the conveyor belt is unavailable and flags a disruption.

The replanning service is implemented as the component for reacting to such anomaly results. Its role is to use the anomaly context to dynamically adapt the production cycle plan. Since we rely on SysML v2 models defining both the structure and behavior of each machine, the recovery action can be localized around the faulty component instead of requiring replanning of the whole production cycle. This machine-level control is supported by the SCADA, which provides services for switching machine missions while allowing unaffected parts of the production cycle to continue running. The anomaly context includes information such as the disrupted step, the affected component, and the current location of the workpiece. Based on this information, the service selects an alternative mission composed of a set of preregistered machine missions that bypass the unavailable component using nearby available resources.

The notification service alerts the maintenance engineer when a disruption is detected and provides the associated context. It is integrated into the existing graphical interface (Fig. 12), which mirrors the factory layout, uses a Spring REST API and server-sent events (SSE) to push disruptions to the frontend, and highlights the faulty machine. The pop-up also lets the maintenance engineer confirm recovery; this event is stored with timestamp, user comment, and reference to the original disruption for later duration and KPI analysis.

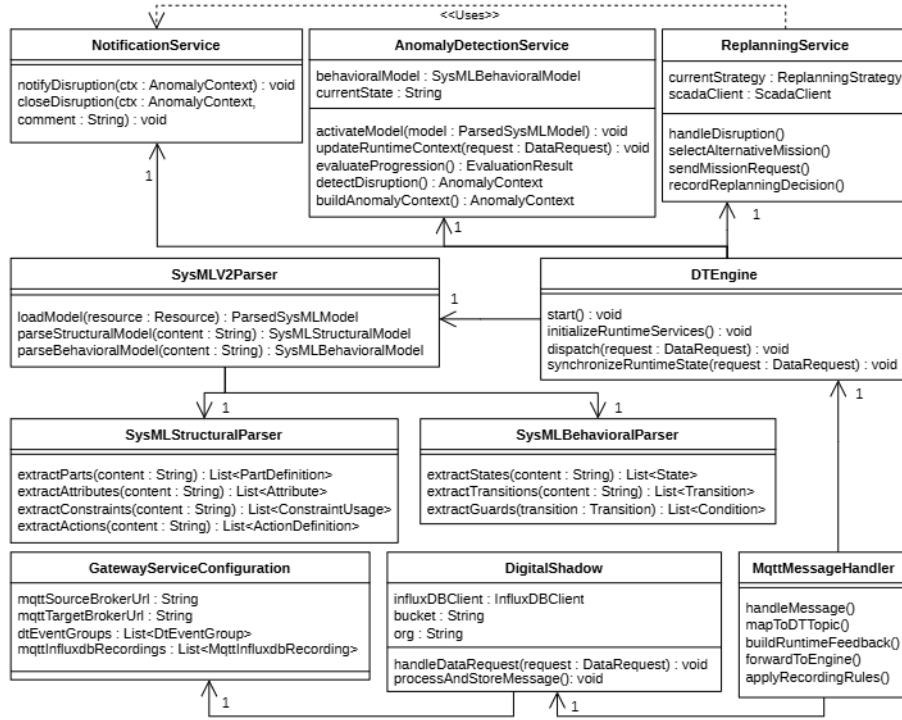


Figure 11: Class diagram of the minimal components needed to build the digital twin for UC2

4.3. Use Case 3: Predictive Maintenance

Motivation. Highly flexible and frequently reconfigured production systems are a response to fast-changing markets, but they also amplify the need for continuous visibility into machine and process state. In such settings, maintenance can no longer rely on purely reactive interventions; instead, data-driven predictive maintenance aims to anticipate failures early, reduce unplanned downtime, and stabilize output quality by intervening before degradation becomes irreversible [16, 17]. Prior work has already demonstrated that self-adaptive digital twins can use operational signals to reason about quality-related indicators and explore parameter spaces under varying conditions (e.g., SDMflex) [18, 19].

In our demonstrator, degradation manifests as increasing deviations in motor speed and cycle timing under comparable operating conditions. Such deviations are commonly driven by cumulative wear and load-dependent effects in motion systems; accuracy degradation is widely attributed to wear mechanisms that accumulate with usage and loading [20, 21]. Moreover, the operating load (here: payload weight) has a direct impact on the dynamic response of conveying systems and thus on observable signals such as transient speed profiles and timing [22]. Consequently, the digital twin monitors speed and timing deviations, predicts whether the trend will reach a failure threshold, and triggers maintenance and recalibration actions before the physical system breaks down.

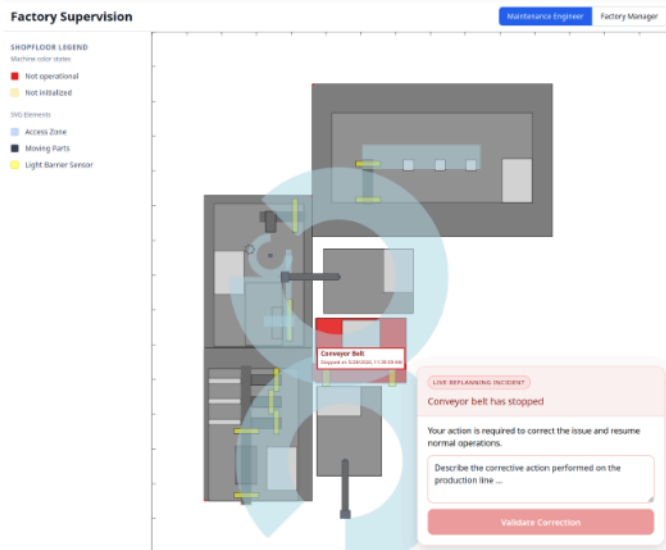


Figure 12: Digital Twin Notification Service GUI

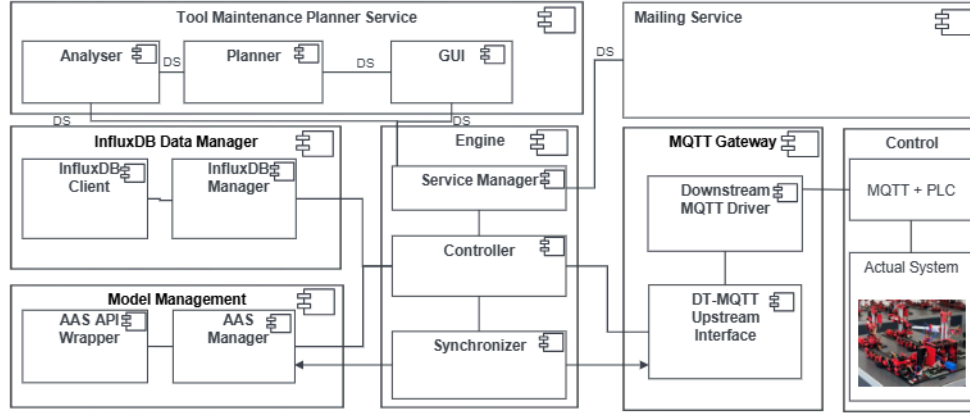


Figure 13: Overview of the Digital Twin Architecture for a Predictive Tool Maintenance Service

Stakeholders. Primary stakeholders are (i) maintenance engineers, who plan and execute interventions, (ii) operators, who observe anomalies and apply operating procedures, (iii) production planners, who coordinate downtime windows, and (iv) quality managers, who relate process stability to product quality and scrap rates.

User stories. **US1** As a maintenance engineer, I want the system to monitor operational speed deviations of the drive motor under different payload weights and duty cycles, so that I can detect early signs of wear or overload and schedule maintenance before a breakdown occurs.

US2 As an operator, I want the system to notify me when abnormal motor behavior occurs and provide guidance on the affected component and recommended handling procedure, so that I can react quickly and operate the line safely.

US3 As a production planner, I want the system to provide predicted maintenance windows and expected intervention durations, so that I can coordinate maintenance activities with production schedules and minimize disruption.

US4 As a quality manager, I want the system to relate motor degradation and maintenance events to product quality indicators and scrap occurrences, so that I can assess whether equipment degradation affects production quality.

Fulfillment Criteria/Requirements.

RQ1 Continuous Logging. The system must continuously record time-series data for the drive motor, including at least motor speed and sensor arrival times for each production cycle, together with timestamps and a unique cycle or part identifier.

RQ2 Context Capture. The system must record the operational context for each production cycle, including payload weight (e.g., normal or increased), operating mode (e.g., idle, normal load, high load), number of start-stop cycles, cumulative operating hours, and production counts, which should be described during design time.

RQ3 Anomaly Detection. The system must derive an expected speed profile for normal operation and detect

anomalies when the observed speed deviates from the expected profile by more than a configurable threshold for the same payload weight and duty cycle.

RQ4 Maintenance Actionability. The system must notify the maintenance engineer when a failure-prone trend is detected and provide a recommended maintenance window together with a concrete intervention, such as recalibration, reset procedures, or mechanical inspection.

RQ5 Operator Actionability. The system must notify the operator when abnormal behavior is detected and provide information about the affected component, the anomaly type, and the recommended immediate handling procedure.

RQ6 Planning Support. The system must estimate the expected duration and urgency of a recommended intervention and make this information available for production planning purposes.

RQ7 Quality Correlation. The system must relate detected anomalies and maintenance events to production outcomes, including quality indicators and scrap, so that the effect of equipment degradation on product quality can be analyzed.

RQ8 Model Support. The system must persist anomaly cases, maintenance events, and associated context data for historical analysis and for the training and evaluation of predictive maintenance models. At least N synthetic anomaly scenarios shall be supported and labeled as “failure-prone” cases.

The maintenance engineer user story requires the system to identify abnormal motor behavior under varying production conditions and to translate these observations into actionable maintenance decisions. This requires continuous operational data acquisition (*RQ1 (Continuous Logging)*), contextualization of each cycle (*RQ2 (Context Capture)*), anomaly detection against expected behavior (*RQ3 (Anomaly Detection)*), and the derivation of concrete maintenance actions (*RQ4 (Maintenance Actionability)*).

The operator user story requires timely visibility of anomalies during plant operation. This means the system must not only detect abnormal behavior (*RQ3 (Anomaly Detection)*) but

also communicate the affected component and suitable handling instructions to support safe and immediate intervention (*RQ5 (Operator Actionability)*).

The production planner user story requires that predictive maintenance results can be incorporated into scheduling decisions. This implies that the system must transform anomaly and maintenance findings into planning-relevant information, especially intervention urgency and expected duration, as specified in *RQ6 (Planning Support)*.

The quality manager user story requires that equipment degradation can be assessed in relation to product quality and scrap. This necessitates linking anomaly and maintenance records to production outcomes and quality indicators, as required by *RQ7 (Quality Correlation)*. Historical persistence of these cases is further required for retrospective analysis and model improvement, as captured in *RQ8 (Model Support)*.

4.3.1. Digital Twin Design to Fulfill the User Stories

To fulfill the user stories introduced above, we implement the predictive maintenance capability shown in Figure 13 on top of the lightweight digital twin architecture introduced in the previous subsection. The cyber-physical system, consisting of PLC-controlled machinery and MQTT-based communication, publishes operational signals to an MQTT broker. The digital twin subscribes to these signals, synchronizes them into its internal representation, persists them, analyzes them, and triggers role-specific responses for maintenance engineers, operators, production planners, and quality managers.

In the concrete realization, the digital twin engine comprises a controller, a synchronizer based on explicit signal-to-model mappings, and a service manager. The MQTT-based ingestion and synchronization pipeline continuously acquires operational data such as motor speed, sensor arrival times, and cycle identifiers (*RQ1*) and enriches them with payload class, operating mode, start-stop frequency, cumulative operating hours, and production counts (*RQ2*).

For persistence, the system stores structured asset and model information in an Asset Administration Shell (AAS) and time-series operational data in an InfluxDB instance. The InfluxDB back end provides the historical storage required for continuous logging and retrospective analysis, directly supporting *RQ1 (Continuous Logging)* and *RQ8 (Model Support)*. The AAS captures asset-related metadata, component descriptions, and contextual information relevant to interpreting operational deviations, thereby contributing to *RQ2 (Context Capture)*. Both persistence back ends are accessed through dedicated management components that expose minimal read and write interfaces to the digital twin engine.

On top of this data foundation, the service manager hosts reusable analysis, planning, notification, and reporting components. An *Analysis* component evaluates logged speed and context data against expected baseline profiles to identify anomalies and failure-prone trends (*RQ3*) and persists labeled anomaly cases for later evaluation and model improvement (*RQ8*).

A *Maintenance Planner* service derives recommendations from detected anomaly patterns, proposes interventions such as

recalibration or inspection, and estimates suitable maintenance windows and durations. It thereby fulfills *RQ4 (Maintenance Actionability)* and *RQ6 (Planning Support)*.

To support operators during runtime, the service layer further provides a notification mechanism that communicates detected abnormal behavior together with the affected component and recommended immediate handling procedures. This realizes *RQ5 (Operator Actionability)* and ensures that anomalies are not only detected analytically but also made operationally visible in a form that supports safe and timely reaction on the shop floor.

Finally, the digital twin exposes historical and aggregated data for quality-oriented assessment by linking anomaly events and maintenance records with quality indicators and scrap occurrences. This realizes *RQ7 (Quality Correlation)* and reinforces *RQ8 (Model Support)*.

Overall, the layered design combines data ingestion, synchronization, analysis, planning, notification, and historical correlation into an integrated basis for predictive maintenance, operational response, production coordination, and quality assessment.

Digital Twin Classes. The digital twin classes are (see Figure 14):

- **DT-MQTT Upstream Interface.** Manages type/object/event/method mappings (typemap, propertymap, eventmap, methodmap) plus the available properties, and exposes the runtime entry points for MQTT-based DT I/O (run, handleEvent, execute).
- **Service Manager.** Orchestrates services via descriptors and bridges, manages provided interfaces/visibility, and coordinates requests and DT events through internal and service event queues (enqueueRequest, notify, register).
- **Synchronizer.** Owns the end-to-end synchronization logic (e.g., MQTT↔AAS property mapping), executes synchronization cycles, and propagates updates to model, data, gateway, service, and workflow components (synchronize, update(...)).
- **Controller.** Runs and controls workflow execution by driving a workflow engine (BPMN) through the main control entry point (run(WorkflowEngine)).
- **AAS Manager.** Manages AAS properties and operations, supports updating AAS elements and path-based access, and handles AAS import/export (update, getElement, import/export).
- **InfluxDB Manager.** Encapsulates time-series persistence around DB column properties and operations, supporting reads/writes, event handling, and client health checks (write, read, handleEvent, health).
- **Analyser.** Ingests observations into an analysis digital shadow, extracts features, computes states/results, and supports reconfiguration of the state model and rule set (ingest, compute, reconfigure).

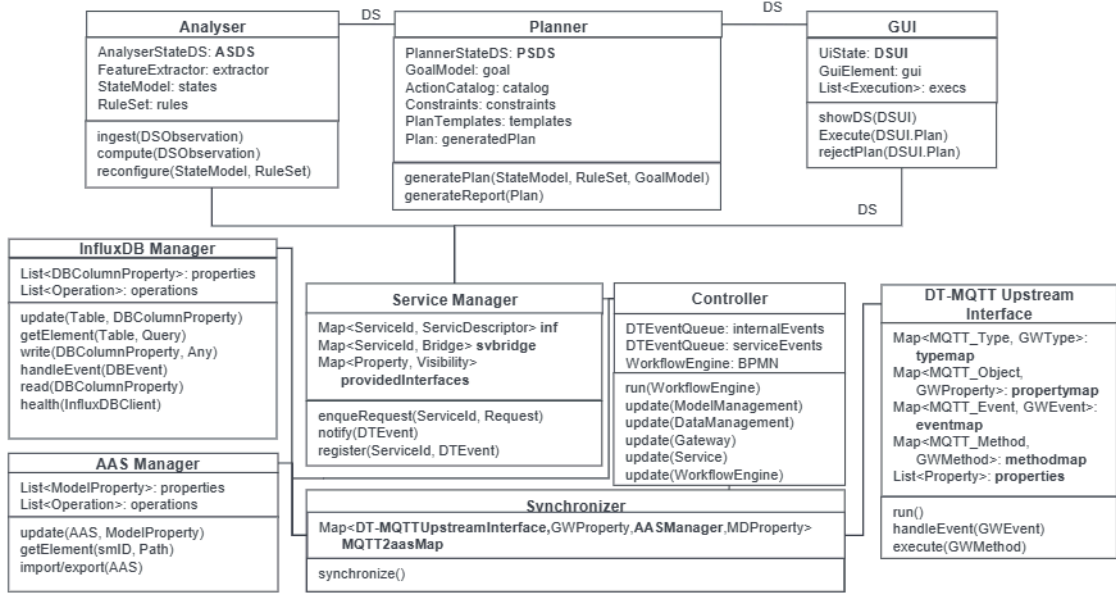


Figure 14: Class diagram of the minimal components to build a digital twin for predictive maintenance.

- **Planner.** Generates a plan from the current state/rules and goals, using an action catalog plus constraints and templates, and can produce a report from the plan (generatePlan, generateReport).
- **GUI.** Presents the digital shadow UI state, visualizes it, and lets a user approve or reject the proposed plan (showDS, Execute, rejectPlan).

4.3.2. Models and Data Used in Analysis & Planning

Predictive maintenance requires linking observed signals to the context that explains them. We therefore represent the machine state with (i) operational time series, (ii) per-cycle aggregates, and (iii) cumulative usage indicators commonly used to estimate how much “life” has been consumed (e.g., hours, cycles, start–stop counts) [17, 16].

Operational and usage data model. For each cycle c , we store:

- **Signals:** motor speed $v(t)$; sensor arrival timestamps $\tau_s(c)$; optionally current/temperature/vibration (if available).
- **Context:** payload class $w(c) \in \{\text{normal, increased}\}$; duty-cycle/mode $m(c)$; start–stop count $\sigma(c)$.
- **Cumulative usage:** operating hours $H(c)$; production count $P(c)$.

Load-dependent effects are explicitly modeled by conditioning the baseline on payload and mode; this mirrors the general observation that loading affects dynamic behavior and thus the expected signal profile [22, 21].

Synthetic failure labels for evaluation. To enable repeatable experiments in the Fischertechnik setup, we create controlled anomaly scenarios by: (i) increasing payload weight (added mass), (ii) increasing friction (e.g., intentional mechanical resistance), or (iii) inducing mild misalignment. All cycles recorded under these conditions are labeled as “failure-prone” to evaluate anomaly detection and simple remaining-useful-life trend estimation.

4.3.3. Adaptation Logic and Maintenance Actions

The analysis component computes (a) per-cycle deviation features and (b) a trend estimate. Deviations are compared against a baseline conditioned on payload and duty cycle; if deviation grows persistently or crosses a threshold, the digital twin plans an intervention. This closes the loop from monitoring to actionable maintenance planning, consistent with MAPE-K-style digital twin integration [23].

Typical intervention policies.

- **Early warning:** deviation exceeds 40% for 3 consecutive cycles $\Rightarrow$ notify maintenance engineer and operator.
- **Safe fallback:** deviation increases sharply under high load $\Rightarrow$ reduce speed setpoint or limit routing and throughput.
- **Recalibration/reset:** if deviation indicates encoder drift or accumulated offset, execute a reset/re-homing procedure and log the effect (before/after comparison).
- **Planned maintenance:** if trend predicts threshold crossing within a horizon (e.g., next T cycles), propose a maintenance window for inspection and readjustment.

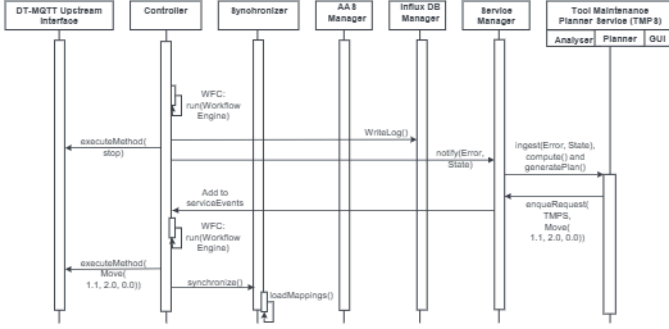


Figure 15: Dataflow in the digital twin when encountering an unknown state or anomaly that triggers analysis and planning.

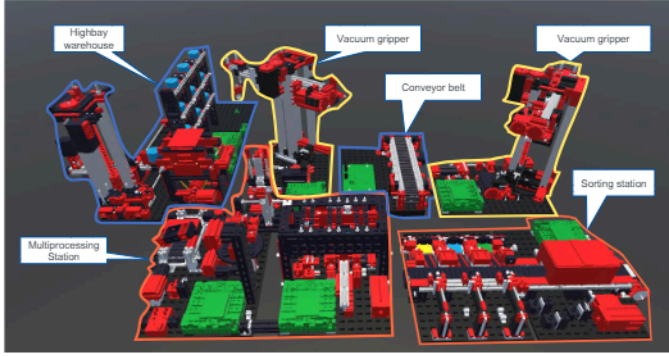


Figure 16: Visualization in User Interface of Demonstrator showing the current state and showing degradation of modules in UI.

4.3.4. Example Implementation of the DT Demonstrator

As illustrated in Figure 15, the runtime follows a closed-loop sequence: the Workflow Controller triggers synchronization and logging, executes a safe fallback action (e.g., `stop`), notifies the planner service with `Error` and `State`, and then executes the planner’s parameterized action request (e.g., `Move(1.1, 2.0, 0.0)`) via the upstream interface.

Signals and MQTT topics. The CPS publishes station signals and process context via the MQTT broker introduced previously. In line with the topic structure used throughout this paper, machine-related topics are published in a machine-specific namespace following the pattern `/${machine-id}/...`. For the predictive-maintenance use case, the digital twin subscribes to a combination of (i) station-level sensor/actuator topics and (ii) process-context topics.

The station-level topics used in UC3 cover arrival and transfer events, motion-related raw signals, derived position signals, and actuation or operating signals. The process-level topics associate each cycle with traceability metadata such as cycle identifiers, payload class, and operating mode.

This combination allows the digital twin to reconstruct cycle timing from sensor events, derive motion-speed-related features from encoder/pulse signals, and evaluate deviations under comparable payload and operating modes.

Feature computation and anomaly detection. On each cycle c , the digital twin aggregates encoder-/pulse-based motion signals and sensor arrivals into cycle-level features. In particular, it derives a speed estimate $\bar{v}(c)$ from the encoder or pulse-counter streams (optionally supported by derived position topics) and combines it with cycle timing features. It then derives

$$\Delta v(c) = \frac{|\bar{v}(c) - \bar{v}_{\text{base}}(w(c), m(c))|}{\bar{v}_{\text{base}}(w(c), m(c))},$$

$$\Delta \tau(c) = \tau_{s2}(c) - \tau_{s1}(c).$$

and flags an anomaly if $\Delta v(c) > X$ (and/or timing residuals exceed a threshold) under the same payload/mode class.

Analysis–planning–execution loop. When an anomaly (or unknown state) is detected, the Workflow Controller (WFC) initiates the loop shown in Figure 15: (i) it calls the Synchronizer (`synchronize()`), which loads the required AAS mappings (AAS Manager: `loadMappings()`) and persists a trace entry (InfluxDB Manager: `WriteLog()`); (ii) the DT-MQTT upstream interface immediately enforces a safe fallback (e.g., `executeMethod(stop)`); (iii) the Service Manager publishes a structured notification containing `Error` and `State` (`notify(Error, State)`); (iv) the maintenance planner service (TMPs) ingests this information (`ingest(Error, State)`), computes a decision (`compute()`), and produces an executable plan (`generatePlan()`); (v) the Service Manager enqueues the resulting parameterized action request into `serviceEvents` (e.g., `enqueueRequest(TMPs, Move(1.1, 2.0, 0.0))`); (vi) the WFC subsequently executes the queued request through the upstream interface (e.g., `executeMethod(Move(1.1, 2.0, 0.0))`).

Outputs. The implementation produces three artifacts aligned with Figure 15: (i) operator-facing alerts via notification (dashboard/event log), as in Figure 16, (ii) persistent anomaly traces and cycle summaries in the time-series database for offline learning (e.g., regression over deviation trends), and (iii) an executable recovery/maintenance action (e.g., `inspect`, `recalibrate/reset`, `schedule downtime`, or a concrete motion/stop command) emitted as a queued service request and executed by the WFC.

5. Discussion

The three use cases show that a manufacturing digital twin should not be understood as a monolithic system for one isolated purpose, but as a composition of physical assets, digital representations, runtime data, synchronization mechanisms, and added-value services. In the Fischertechnik demonstrator, the same production modules are reused across different concerns: energy monitoring treats machines as energy-consuming resources, flexible replanning interprets them as state-dependent production resources, and predictive maintenance considers them as components whose behavior may indicate degradation. This shows that digital twin composition requires multiple complementary views on the same physical system rather than a single universal model.

A central lesson is that architectural reuse is possible, but only if the interaction between services is explicitly considered. The three use cases rely on recurring architectural elements, including gateways, synchronizers, data managers, model managers, and service components. However, reuse becomes more challenging when services not only observe the system but also influence it. For example, a replanning service may trigger an alternative mission, while a maintenance service may recommend slowing down or stopping a component. Such cases show that composed digital twins require coordination mechanisms, service contracts, and clear access rules for shared models, data, and control functions.

This has direct implications for the engineering of digital twin systems. Before implementation, engineers need to define the digital twin boundary, identify twinable properties, specify their semantics, model the relevant structural and behavioral views, and define mappings between physical signals, digital shadows, models, and service inputs. In addition, composed digital twins require explicit service contracts that describe required inputs, produced outputs, possible actions, priorities, and side effects. Thus, current digital twin engineering should place stronger emphasis on preparing reusable architectural and model-based foundations, instead of developing each digital twin service as an isolated use-case-specific solution.

Limitations. The presented digital twin exemplar has several limitations. First, the evaluation is based on a single Fischertechnik production line, which, although representative for a small-scale production process, cannot capture the complexity and heterogeneity of industrial production systems. Second, the three use cases demonstrate the flexibility of the proposed architecture but do not provide a quantitative comparison with alternative architectures or implementations. Third, the predictive maintenance scenario relies on simulated degradation data rather than real machine failures, limiting conclusions about its effectiveness in industrial environments. Finally, while the architecture is designed to be extensible, its scalability to substantially larger production systems and its interoperability with heterogeneous industrial infrastructures have not yet been evaluated. Addressing these limitations through industrial case studies, larger-scale deployments, and quantitative performance evaluations constitute important future work.

6. Conclusion

This paper presented a digital twin case study for manufacturing systems based on a modular Fischertechnik factory demonstrator. The aim was to move beyond abstract digital twin definitions and reference models by showing how a manufacturing digital twin architecture can be instantiated for a concrete, executable production setup. For this purpose, we described the physical system, its control devices, communication interfaces, and twinable properties, and mapped them to a digital twin architecture consisting of gateways, a digital twin engine, synchronization mechanisms, data and model management components, and added-value services. The case study was investigated through three operational use cases. The presented demonstrator provides a reusable basis for comparing

digital twin engineering approaches in a controlled manufacturing setting.

Acknowledgements

Funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) - Model-Based DevOps - 505496753. Funded by the Agence Nationale De La Recherche (ANR) - France. Website: <https://mbdo.github.io>.

Generative AI Disclosure

During the preparation of this work the authors used ChatGPT and Gemini in order to check grammar and reformulate text. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

- [1] F. Tao, Q. Qi, L. Wang, A. Y. C. Nee, Digital twins and cyber-physical systems toward smart manufacturing and industry 4.0: Correlation and comparison, *Engineering* 5 (4) (2019) 653–661. doi:10.1016/j.eng.2019.01.014.
- [2] A. Fuller, Z. Fan, C. Day, C. Barlow, Digital twin: Enabling technologies, challenges and open research, *IEEE Access* 8 (2020) 108952–108971. doi:10.1109/ACCESS.2020.2998358.
- [3] W. Kritzing, M. Karner, G. Traar, J. Henjes, W. Sih, Digital twin in manufacturing: A categorical literature review and classification, *IFAC-PapersOnLine* 51 (11) (2018) 1016–1022. doi:10.1016/j.ifacol.2018.08.474.
- [4] J. Michael, L. Cleophas, S. Zschaler, T. Clark, B. Combemale, T. Godfrey, D. E. Khelladi, V. Kulkarni, D. Lehner, B. Rumpe, M. Wimmer, A. Wortmann, S. Ali, B. Barn, I. Barosan, N. Bencomo, F. Bordeleau, G. Grossmann, G. Karsai, O. Kopp, B. Mitschang, P. Muñoz Ariza, A. Pierantonio, F. A. C. Polack, M. Riebisch, H. Schlingloff, M. Stumptner, A. Vallecillo, M. van den Brand, H. Vangheluwe, Model-Driven Engineering for Digital Twins: Opportunities and Challenges, *Systems Engineering* 28 (5) (2025) 659–670. doi:10.1002/sys.21815.
- [5] J. Pfeiffer, J. Zhang, B. Combemale, J. Michael, B. Rumpe, M. Wimmer, A. Wortmann, Towards a Unifying Reference Model for Digital Twins of Cyber-Physical Systems, in: 2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA), 2025, pp. 1–4. doi:10.1109/ETFA65518.2025.11205646.

- [6] International Organization for Standardization, ISO 23247-1:2021: Automation systems and integration – Digital twin framework for manufacturing – Part 1: Overview and general principles, <https://www.iso.org/standard/75066.html> (2021).
- [7] T. A. Abdel-Aty, E. Negri, S. Galparoli, Asset administration shell in manufacturing: Applications and relationship with digital twin, *IFAC-PapersOnLine* 55 (10) (2022) 2533–2538. doi:10.1016/j.ifacol.2022.10.090.
- [8] J. Zhang, C. Ellwein, M. Heithoff, J. Michael, A. Wortmann, Digital Twin and the Asset Administration Shell: An Analysis of 3 AASs Types and their Feasibility for Digital Twin Engineering, *Journal Software and Systems Modeling (SoSyM)* 24 (2025) 771–793. doi:10.1007/s10270-024-01255-0.
- [9] OASIS Message Queuing Telemetry Transport (MQTT) Technical Committee, MQTT Version 5.0, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>, oASIS Standard, 7 March 2019 (Mar. 2019).
- [10] R. van Dinter, B. Tekinerdogan, C. Catal, Predictive maintenance using digital twins: A systematic literature review, *Information and Software Technology* 151 (2022) 107008. doi:10.1016/j.infsof.2022.107008.
- [11] H. Gurocak, *Industrial motion control: motor selection, drives, controller tuning, applications*, John Wiley & Sons, 2015.
- [12] DIN Deutsches Institut für Normung, *Maintenance – Qualification of maintenance personnel* (Oct. 2014).
- [13] A. Haber, J. O. Ringert, B. Rumpe, MontiArc - Architectural Modeling of Interactive Distributed and Cyber-Physical Systems, Technical Report AIB-2012-03, RWTH Aachen University (February 2012).
- [14] C. Buschhaus, A. Gerasimov, J. C. Kirchhof, J. Michael, L. Netz, B. Rumpe, S. Stüber, Lessons Learned from Applying Model-Driven Engineering in 5 Domains: The Success Story of the MontiGem Generator Framework, *Science of Computer Programming* 232 (2024) 103033. doi:10.1016/j.scico.2023.103033.
- [15] M. Dalibor, N. Jansen, J. C. Kirchhof, B. Rumpe, D. Schmalzing, A. Wortmann, Tagging Model Properties for Flexible Communication, in: N. Ferry, A. Cicchetti, F. Cicciozzi, A. Solberg, M. Wimmer, A. Wortmann (Eds.), *Proceedings of MODELS 2019. Workshop MDE4IoT, CEUR Workshop Proceedings*, 2019, pp. 39–46.
- [16] P. Mallioris, E. Aivazidou, D. Bechtsis, Predictive maintenance in industry 4.0: A systematic multi-sector mapping, *CIRP journal of manufacturing science and technology* 50 (2024) 80–103.
- [17] A. Theissler, et al., Predictive maintenance enabled by machine learning, *Reliability Engineering & System Safety* PII: S0951832021003835 (2021).
- [18] SDM4FZI Project Consortium, Software-defined Manufacturing für die Fahrzeug- und Zulieferindustrie, <https://www.sdm4fzi.de/>, research project SDM4FZI (2023).
- [19] InnovationsCampus Mobilität der Zukunft (ICM), Sd-Manu4 - SDMflex: Flexible SDM through Continuously Quality-Aware Digital Twins (2024). URL <https://www.icm-bw.de/forschung/projektuebersicht/detailseite/sdmanu4-sdmflex>
- [20] Q. Cheng, et al., An accuracy degradation analysis of ball screw mechanism considering the time-varying motion and loading working conditions, *Mechanism and Machine Theory* ScienceDirect (2019).
- [21] J. Zhao, et al., Research on the precision loss of ball screw with short-time heavy load operation, *Advances in Mechanical Engineering* (2018). doi:10.1177/1687814018817902.
- [22] D. He, Y. Pang, G. Lodewijks, Speed control of belt conveyors during transient operation, *Powder Technology* (2016). doi:10.1016/j.powtec.2016.07.004.
- [23] H. Feng, C. Gomes, S. Gil, P. H. Mikkelsen, D. Tola, P. G. Larsen, M. Sandberg, Integration of the Mape-K Loop in Digital Twins, *ANNSIM '22, Society for Computer Simulation International*, San Diego, CA, USA, 2022. doi:10.22360/ANNSIM.2022.CPS.92.